FANTASYREALM ONLINE

Documentation technique

Choix technologiques, environnement, données, diagrammes, sécurité et déploiement

1. Réflexions technologiques initiales

L'énoncé impose une base relationnelle et une base non relationnelle et laisse le reste libre. Mes critères : pouvoir **expliquer chaque ligne** devant le jury, couvrir toutes les compétences du titre (front, back, SQL, NoSQL, déploiement), et produire une application légère, sécurisée et accessible.

Besoin	Choix	Justification	Alternative écartée
Front-end	HTML5, CSS3, JavaScript natif	Aucune dépendance, pages légères (éco-conception), maîtrise totale de l'accessibilité. L'interactivité nécessaire (aperçu, contrôles de saisie) reste modeste.	Bootstrap (proposé par l'énoncé) : plus lourd, visuel générique ; React : inutile pour des pages surtout serveur.
Back-end	PHP 8.3 sans framework, MVC maison	Langage du cursus, hébergement simple et peu coûteux, chaque mécanisme (routeur, sessions, CSRF) est écrit et donc expliqué.	Symfony : très complet mais masque ces mécanismes ; Node.js : hébergement moins courant en mutualisé.
Accès SQL	PDO, requêtes préparées	Natif, protège de l'injection SQL, portable (MySQL, MariaDB, PostgreSQL).	ORM Doctrine : abstraction qui cache le SQL que l'ECF demande de maîtriser.
Base relationnelle	MySQL 8	Données fortement liées (comptes, personnages, équipement, avis) : clés étrangères, contraintes UNIQUE et CHECK, transactions.	PostgreSQL : équivalent, moins répandu chez les hébergeurs mutualisés.
Base NoSQL	MongoDB 7	Logs de personnalisation : écrits souvent, sans relations, de structure variable selon l'événement ; agrégations faciles pour les statistiques.	Redis : orienté cache, peu adapté à la consultation d'un historique.
Images des personnages	SVG généré côté serveur	Pas de bibliothèque d'images ni de fichiers à stocker, rendu net à toute taille, quelques ko par avatar, aperçu instantané.	Images PNG superposées : lourdes, nombreuses ressources graphiques à produire.
E-mails	PHPMailer (SMTP)	Bibliothèque de référence, gère l'encodage et le TLS, protège des injections d'en-têtes.	Fonction mail() : non authentifiée, souvent bloquée.
Configuration	vlucas/phpdotenv	Secrets dans un fichier `.env` hors du dépôt Git.	Constantes en dur : fuite des mots de passe dans Git.
Tests	PHPUnit 11	Standard PHP.	-

2. Environnement de travail

- **Docker Desktop** et `docker-compose.yml` : cinq conteneurs (Apache + PHP 8.3, MySQL 8, MongoDB 7, Mailpit pour intercepter les e-mails, phpMyAdmin). Toute personne qui clone le dépôt obtient exactement le même environnement, sans rien installer d'autre que Docker. Les versions sont celles de la production.
- **Composer** pour les dépendances PHP (exécuté dans le conteneur).
- **Git** et GitHub, avec les branches `main`, `develop` et `feature/…`.
- **Éditeur de code** : [À PERSONNALISER : VS Code, PhpStorm…], terminal, navigateur et ses outils de développement.
- **Assistant IA** : Claude (Anthropic) dans l'application de bureau Claude Code, utilisé pour l'ensemble du projet.

- **Node.js** : uniquement pour les scripts de documentation (captures et PDF via Edge en mode headless, dossier `docs/outils`).

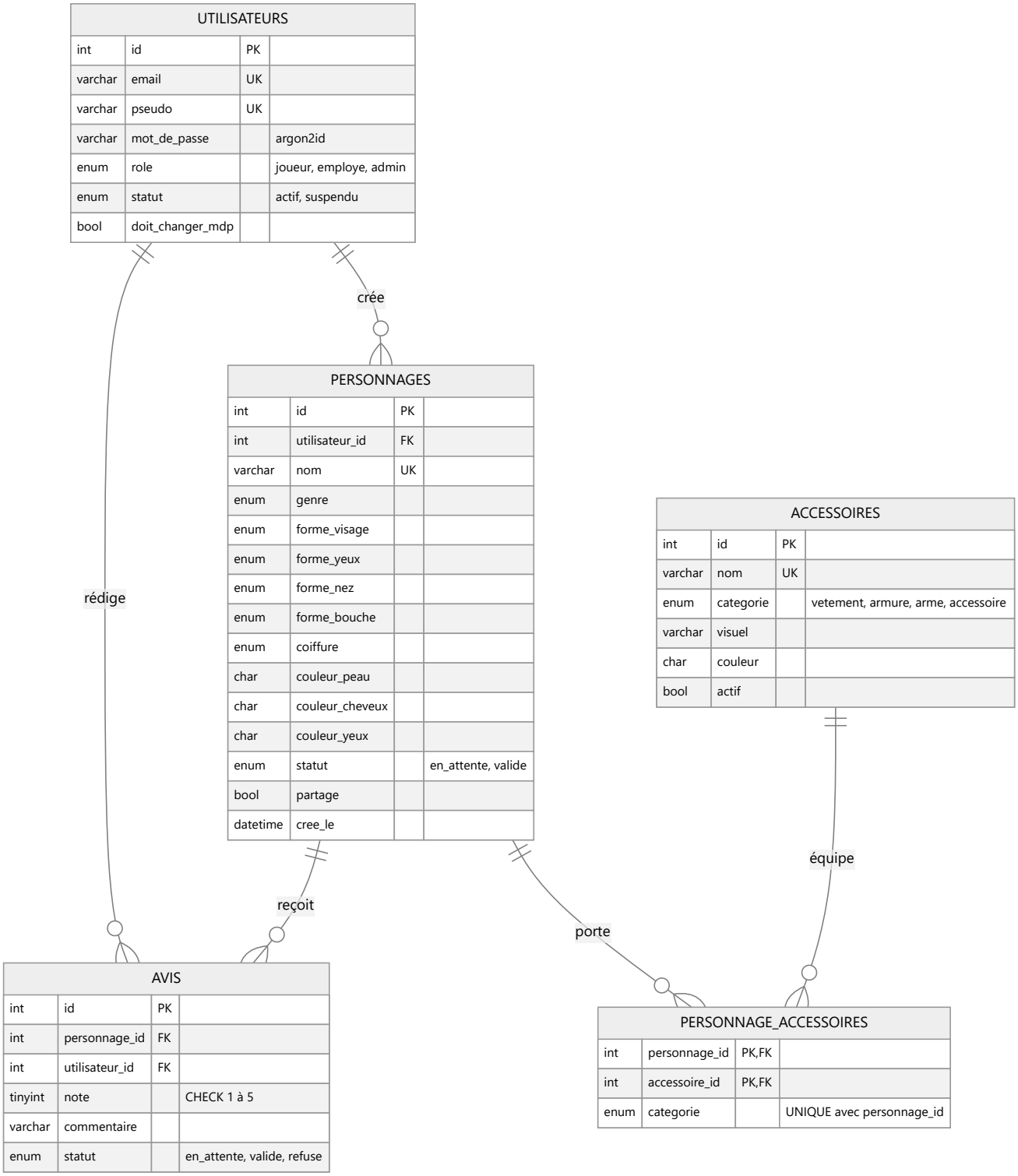
```
docker compose up -d --build                # construit et démarre les conteneurs
docker compose exec app composer install    # dépendances PHP
docker compose exec app vendor/bin/phpunit  # tests
```

3. Architecture

Architecture **MVC** légère. Seul le dossier `public/` est exposé par le serveur web : le code, les vues et le fichier `.env` ne sont jamais téléchargeables.

```
public/index.php      point d'entrée unique : configuration, en-têtes de sécurité, session, déclaration des routes
src/Core/             Router (CSRF + rôles), Auth (sessions), Csrf, Validator, Security, Database (PDO), Mongo, View, Flash
src/Controllers/      Page, Auth, Contact, Galerie, Espace (joueur), Gestion (employé), Admin
src/Models/           Utilisateur, Personnage, Accessoire, Avis, TentativeConnexion (MySQL), MessageContact (MongoDB)
src/Services/         Apparence (options + rendu SVG), ActivityLog (MongoDB), MailService, MotDePasse
templates/            gabarit commun et vues
database/             01_schema.sql (création), 02_donnees.sql (données de démonstration)
tests/                tests PHPUnit
```

4. Modèle conceptuel de données



Règles de gestion traduites dans le schéma

- E-mail, pseudo, nom de personnage et nom d'article uniques : contraintes `UNIQUE`.
- Un seul article par catégorie sur un personnage : `UNIQUE (personnage_id, categorie)` dans la table d'association.
- Un seul avis par joueur et par personnage : `UNIQUE (personnage_id, utilisateur_id)` ; note de 1 à 5 : `CHECK`.
- Un personnage ne peut être partagé que s'il est validé : `CHECK (partage = FALSE OR statut = 'valide')`.

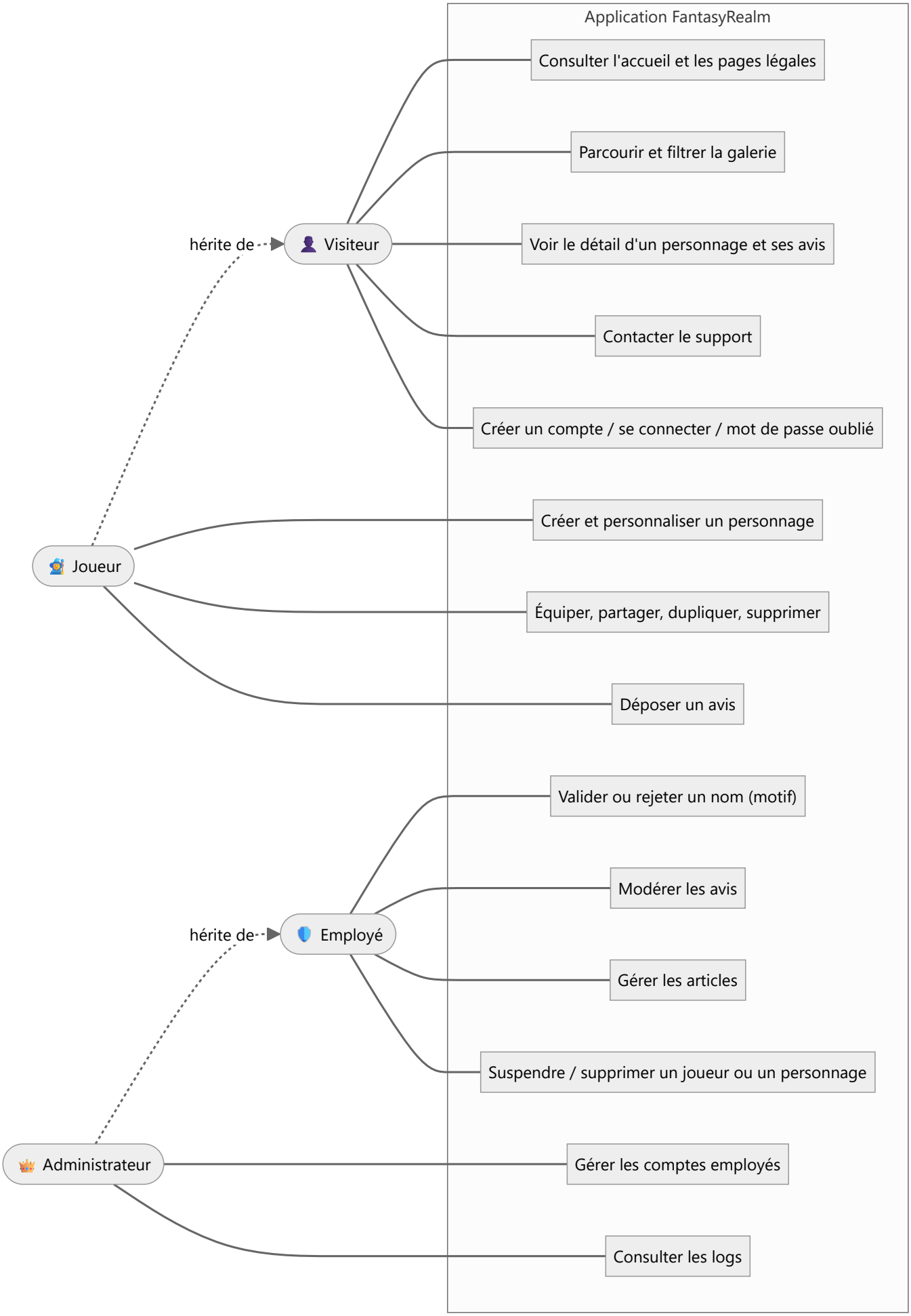
- Supprimer un compte supprime ses personnages, leur équipement et les avis (`ON DELETE CASCADE`).
- Traits physiques en `ENUM` : la base refuse toute valeur hors liste, en plus du contrôle PHP.
- Le compte administrateur est créé par le script SQL : aucune fonctionnalité ne permet d'en créer un.

Collections MongoDB

Collection	Exemple de document
`logs`	`{ type: "personnage.equipement", acteur: { id: 3, pseudo: "Lyra" }, personnage: { id: 1, nom: "Aelwen" }, details: { changements: ["Arme : Épée runique → Arc sylvestre"] }, ip: "…", cree_le: ISODate() }`
`messages`	`{ email, pseudo, demande, cree_le }` : trace des demandes de contact

Le nom du personnage et le pseudo sont recopiés dans le log : l'historique reste lisible même après un renommage ou une suppression.

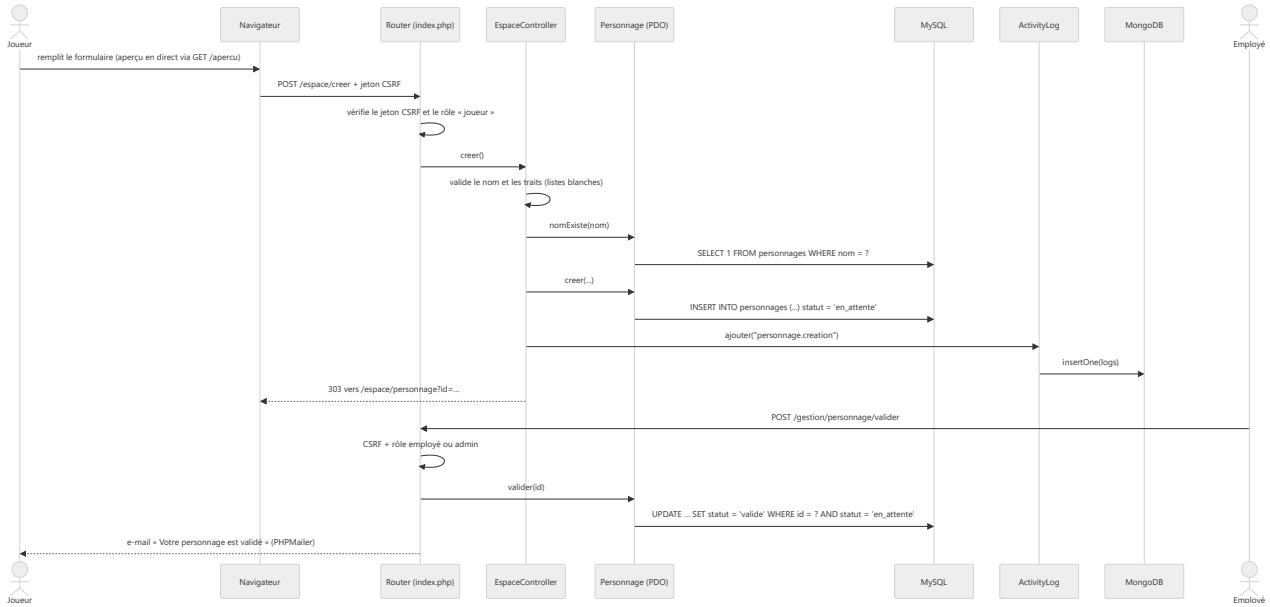
5. Diagramme de cas d'utilisation


Application FantasyRealm
Consulter l'accueil et les pages légales
Parcourir et filtrer la galerie
hérite de
Visiteur
Voir le détail d'un personnage et ses avis
Contacter le support
Créer un compte / se connecter / mot de passe oublié
Créer et personnaliser un personnage
Joueur
Équiper, partager, dupliquer, supprimer
Déposer un avis
Valider ou rejeter un nom (motif)
Modérer les avis
hérite de
Employé
Gérer les articles
Suspendre / supprimer un joueur ou un personnage
Administrateur
Gérer les comptes employés
Consulter les logs

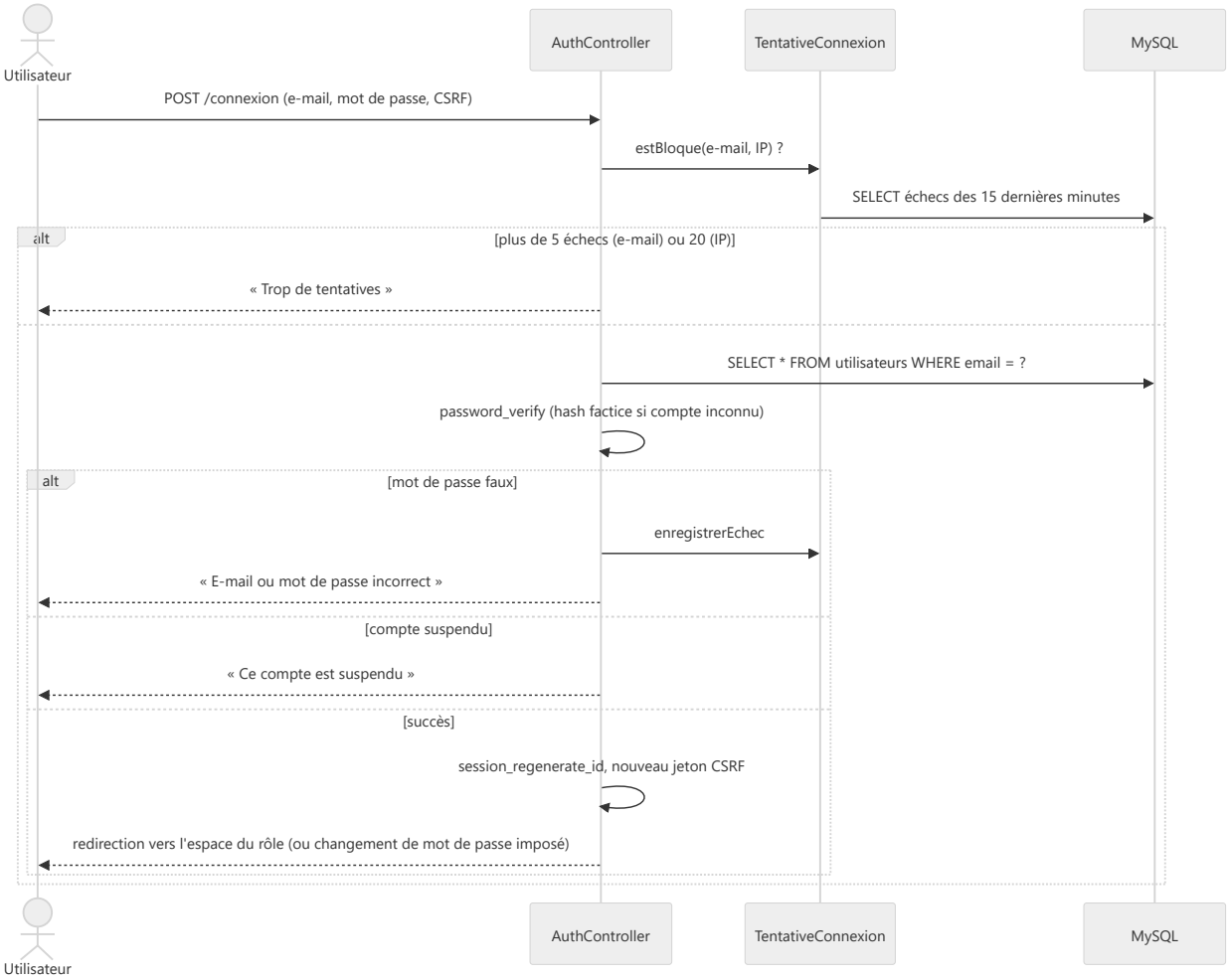
Le joueur peut faire tout ce que fait un visiteur ; l'administrateur tout ce que fait un employé.

6. Diagrammes de séquence

Création d'un personnage puis validation de son nom



Connexion


Utilisateur
AuthController
TentativeConnexion
MySQL
POST /connexion (e-mail, mot de passe, CSRF)
estBloque(e-mail, IP) ?
SELECT échecs des 15 dernières minutes
alt
[plus de 5 échecs (e-mail) ou 20 (IP)]
« Trop de tentatives »
SELECT * FROM utilisateurs WHERE email = ?
password_verify (hash factice si compte inconnu)
alt
[mot de passe faux]
enregistrerEchec
« E-mail ou mot de passe incorrect »
[compte suspendu]
« Ce compte est suspendu »
[succès]
session_regenerate_id, nouveau jeton CSRF
redirection vers l'espace du rôle (ou changement de mot de passe imposé)
Utilisateur
AuthController
TentativeConnexion
MySQL

7. Sécurité (résumé)

Risque	Protection
Injection SQL	PDO, requêtes préparées, `EMULATE_PREPARES = false`, noms de colonnes jamais issus de la saisie
Injection NoSQL	Filtres MongoDB construits avec des valeurs typées (chaînes), expressions régulières échappées
XSS	Échappement systématique (`e()`), SVG construit à partir de listes fermées, CSP sans script en ligne
CSRF	Jeton vérifié par le routeur pour toute requête POST, cookie `SameSite=Lax`
Mots de passe	Règles CNIL, hash argon2id, mot de passe provisoire aléatoire (`random_int`) à changer obligatoirement
Force brute	5 échecs par e-mail ou 20 par IP en 15 minutes
Sessions	Cookie HttpOnly / Secure, régénération à la connexion, expiration, compte suspendu déconnecté immédiatement
Contrôle d'accès	Rôle exigé déclaré sur chaque route ; un joueur n'agit que sur ses personnages ; rôle imposé dans les requêtes de suspension
Données invalides	Validation serveur de chaque champ, refus des requêtes non UTF-8 (400)
Fuite d'informations	Erreurs masquées en production, version d'Apache/PHP masquée, messages neutres (connexion, mot de passe oublié)

8. Déploiement

Démarche. L'application a besoin de PHP 8.3 avec l'extension MongoDB, d'une base MySQL, d'une base MongoDB et d'un serveur SMTP, le tout en HTTPS. J'ai retenu **alwaysdata** (hébergeur français : PHP, MySQL, SMTP, SSH et certificat Let's Encrypt, offre gratuite de 100 Mo) et **MongoDB Atlas** (offre gratuite M0, hébergée en Europe). Heroku et fly.io, cités dans l'énoncé, demandent une carte bancaire ou une image Docker à maintenir ; Vercel ne fait pas tourner PHP nativement.

Étapes

1. **MongoDB Atlas** : créer un cluster M0 (région Europe), un utilisateur limité au rôle `readWrite` sur la base `fantasyrealm`, autoriser l'IP du serveur, copier la chaîne `mongodb+srv://…`.
2. **alwaysdata** : créer le compte, choisir PHP 8.3, vérifier l'extension `mongodb` (sinon `pecl install mongodb`), créer la base MySQL et son utilisateur, créer le site avec pour racine le dossier `public/`, activer HTTPS et la redirection.
3. **Code** (en SSH) :

   ```
   git clone https://github.com/<compte>/fantasyrealm.git
   cd fantasyrealm
   composer install --no-dev --optimize-autoloader
   cp .env.example .env && nano .env      # APP_ENV=prod, accès MySQL, MongoDB, SMTP
   chmod 600 .env
   ```

4. **Base de données** : importer `database/01_schema.sql` puis `database/02_donnees.sql` (données de démonstration utiles au jury) avec phpMyAdmin ou `mysql -h … -u … -p base < database/01_schema.sql`.
5. **Vérifications** : HTTPS actif, `/.env` et `/src/` inaccessibles (404), en-têtes de sécurité présents (`curl -I`), parcours complet avec les comptes de test, e-mails reçus, logs visibles dans l'administration.
6. **Mises à jour** : fusion dans `main`, puis `git pull` et `composer install --no-dev` sur le serveur.

> [À COMPLÉTER après la mise en ligne : URL de production, captures de la configuration, éventuelles difficultés rencontrées.]